RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science & Engineering

CSE 3206: Software Engineering Sessional (22 Series)

LEARNING LOG: SELF-DIRECTED LIFELONG LEARNING (CO4, PO12)

Project: CampusEvents — RUET Campus Event Management System

Student Name	Roll / ID	New Tool Studied	Academic Series
Badruddin Tasnim	2203001	GitHub Projects (Kanban & Automation)	22 Series (RUET CSE)

1. Objectives & Overview of Self-Directed Learning

In fulfillment of Course Outcome CO4 and Program Outcome PO12 (Recognizing the need for, and having the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change), this Learning Log documents the self-directed acquisition of modern engineering methodologies outside the core sessional curriculum. Specifically, it chronicles the adoption of GitHub Projects for Agile sprint management, Test-Driven Development (TDD) using Vitest with external dependency mocks, and the practical implementation of GoF Design Patterns.

2. Chronological Engineering Log & Sprint Milestones

Date & Sprint	Focus Area / Task	Self-Directed Activity & Tool Application	Key Engineering Takeaway
Day 1-3 (Sprint 0)	Tool Selection & Environment Setup	Researched project management alternatives beyond the syllabus. Compared Jira vs. Trello vs. GitHub Projects. Configured GitHub Projects beta with custom Kanban status fields (Backlog, Ready, In Progress, In Review, Done).	Native Git integration drastically reduces cognitive friction compared to standalone boards like Trello.
Day 4-6 (Sprint 1)	Requirements & User Stories	Drafted formal functional requirements for 5 modules. Converted user stories into GitHub Issues with Acceptance Criteria (Given-When-Then). Linked issues to the GitHub Project board.	Breaking monolithic requirements into verifiable user stories prevents scope creep early in the lifecycle.
Day 7-9 (Sprint 1)	Design Patterns Architecture	Studied GoF creational and behavioral patterns. Implemented Singleton pattern for Firebase initialization, Factory pattern for polymorphic campus events, and Observer pattern for event bus notifications.	Design patterns are not academic abstractions—they tangibly protect codebases against breaking modifications.
Day 10-12 (Sprint 2)	TDD & Mocking Strategy	Transitioned away from cumbersome legacy testing setups. Self-learned Vitest and v8 coverage. Implemented isolated mocks for Firebase Firestore methods (`getDocs`, `setDoc`, `updateDoc`).	Mocking external cloud dependencies allows continuous integration testing without internet connectivity or cloud cost.
Day 13-14 (Sprint 2)	Git Workflow & Merge Conflict	Enforced strict feature-branch naming. Created PR #1 and PR #2 with simulated peer reviews. Induced a deliberate merge conflict on validation logic and resolved it cleanly.	Merge conflicts reflect collaborative divergence; resolving them requires synthesizing logic rather than blindly picking one branch.
Day 15-16 (Sprint 3)	UI Refinement & Extensibility	Constructed responsive UI with Tailwind CSS and Lucide icons. Subscribed UI components to EventBus for real-time seat decrementing and ticket badge updates.	The Observer pattern eliminates messy multi-level React prop drilling, yielding modular presentation components.

3. Deep Technical Challenges & Resolution Log

Challenge 1: Cloud Firestore Network Coupling in Offline Test Suites

- **Problem Description:** When executing automated unit tests, test suites attempted real HTTP connections to Firebase Firestore, failing when network latency spiked or during offline campus lab sessions.
- **Root Cause & Resolution:** Learned and applied Vitest module mocking via `vi.mock('firebase/firestore')`. Intercepted raw SDK queries with in-memory deterministic responses and engineered a dual-mode persistence fallback in `EventService` utilizing local storage.
- **Engineering Lesson:** Decoupling domain services from persistence implementations is imperative for reproducible testing.

Challenge 2: State Synchronization Across Decoupled Components

- **Problem Description:** Upon reserving an event ticket in the details modal, the card seat quota on the home page and the navbar ticket count badge failed to reflect the updated count without manually refreshing the browser.
- **Root Cause & Resolution:** Designed and implemented a centralized, lightweight publish-subscribe `EventBus` (Observer pattern). Wrapped subscriptions inside React's `useEffect` hook with dedicated unsubscriber cleanups, allowing components to reactively re-render upon `RSVP_CONFIRMED` broadcasts.
- **Engineering Lesson:** Behavioral patterns provide clean, scalable event pipelines that preserve component decoupling.

Challenge 3: TDD Edge-Case Failure in Factory Validation

- **Problem Description:** During TDD execution of `EventFactory.test.js`, the test verifying minimum sports team sizes failed unexpectedly when passed `teamSize: 0`.
- **Root Cause & Resolution:** The constructor used JavaScript's short-circuit fallback: `this.teamSize = Number(data.teamSize) || 1`. Because `0` is falsy in JavaScript, the expression coerced valid zero-input to 1, causing validation to pass mistakenly. Corrected the expression to explicit existence checking: `data.teamSize !== undefined ? Number(data.teamSize) : 1`.
- **Engineering Lesson:** Writing assertions prior to implementation exposes subtle language-level type coercion traps before code enters production.

4. Critical & Honest Evaluation of Learning Resources

Learning Resource	Category	Strengths Observed	Weaknesses & Limitations	Rating
Official GitHub Documentation & GitHub Skills	Documentation	Authoritative, clear step-by-step interactive labs on GitHub Projects automation, project fields, and PR workflows.	Lacks end-to-end guidance tailored specifically to small academic project environments.	4.5/5.0
Refactoring.Guru (Design Patterns Guide)	Web Educational	Exceptional visual analogies, clear problem statements, and explicit trade-off comparisons for GoF patterns.	Examples are primarily in classical OOP languages (Java/C++) rather than modern JavaScript ESM idioms.	4.8/5.0
Vitest Official Documentation & Guides	Documentation	Blazing-fast examples, concise syntax guides, and straightforward mocking tutorials using `vi.mock()`.	Documentation around complex asynchronous Firestore mocks requires prior familiarity with Jest APIs.	4.2/5.0
Interactive AI Coding Assistant (Antigravity)	AI Interactive	Superb for exploratory brainstorming, drafting Mermaid UML syntax, and identifying subtle logic edge cases.	Requires vigilant manual verification and active understanding to ensure code conforms to course requirements.	4.7/5.0